Repo Summary

Project Divert

One-page app summary generated from repository evidence only.

What It Is

Project Divert is a Flask-based waste and materials platform with an Expo mobile app. Repo evidence shows server-rendered workflows, JSON APIs, dispatch, compliance, billing, and realtime request tracking for waste-removal operations.

Who It's For

Primary users appear to be customers booking waste-removal jobs, drivers/carriers fulfilling them, and admin/ops teams managing dispatch, compliance, auth security, and billing.

What It Does

- Lists reusable materials, supports search/filter views, and accepts material requests.
- Captures waste-removal bookings with pickup details, scheduling, notes, and match radius.
- Matches nearby providers and creates ranked dispatch offers based on distance and quality signals.
- Provides auth APIs for login, signup, refresh, email verification, password reset, and logout.
- Streams request events and latest vehicle location for live customer and driver updates.
- Tracks compliance documents for requests, drivers, and carrier companies, including admin review.
- Supports billing workflows, Stripe-backed charges/refunds/payouts, and push subscription APIs.

How It Works

- **Frontend/UI:** Flask templates under `templates/` for home, materials, maps, auth, admin dispatch, and request forms; React Native/Expo client in `mobile-app/`.
- **Backend:** Single Flask app in `app.py` with HTML routes and `/api/v1/*` endpoints.
- **Data layer:** SQLAlchemy models with Alembic migrations; Postgres configured via `SQLALCHEMY_DATABASE_URI` / `DATABASE_URL`.
- **Reference data:** Pandas-backed supplier, site, and carbon-offset data loaded from repo CSV/XLSX files and seedable via `flask seed-reference-data`.
- **Services/integrations:** Google Maps geocoding/drive-time, SendGrid-compatible email, optional Redis rate limiting, optional S3 compliance storage, Stripe payments, Expo push.
- **Flow:** Customer submits request -> backend stores job + provider matches/offers -> driver/admin accepts and updates status/location -> SSE and push notify mobile clients -> admin reviews compliance/billing.

How To Run

- **Backend:** `pip install -r requirements.txt`
- **Config:** set env from `.env.example`; minimum repo-documented values are `SECRET_KEY`, `DATABASE_URL`, and `GOOGLE_MAPS_API_KEY`.
- **Database:** `flask db upgrade` then `flask seed-reference-data`
- **Run web/API:** `python app.py` locally, or `gunicorn app:app` for deploys
- **Mobile (optional):** in `mobile-app/`, copy `.env.example`, set `EXPO_PUBLIC_API_BASE_URL`, then run `npm install` and `npm run start`.

Evidence sources inspected: app.py, config.py, forms.py, tests/test_app_smoke.py, mobile-app/README.md, mobile-app/App.tsx, render.yaml, DEPLOY.md, and repo file structure.